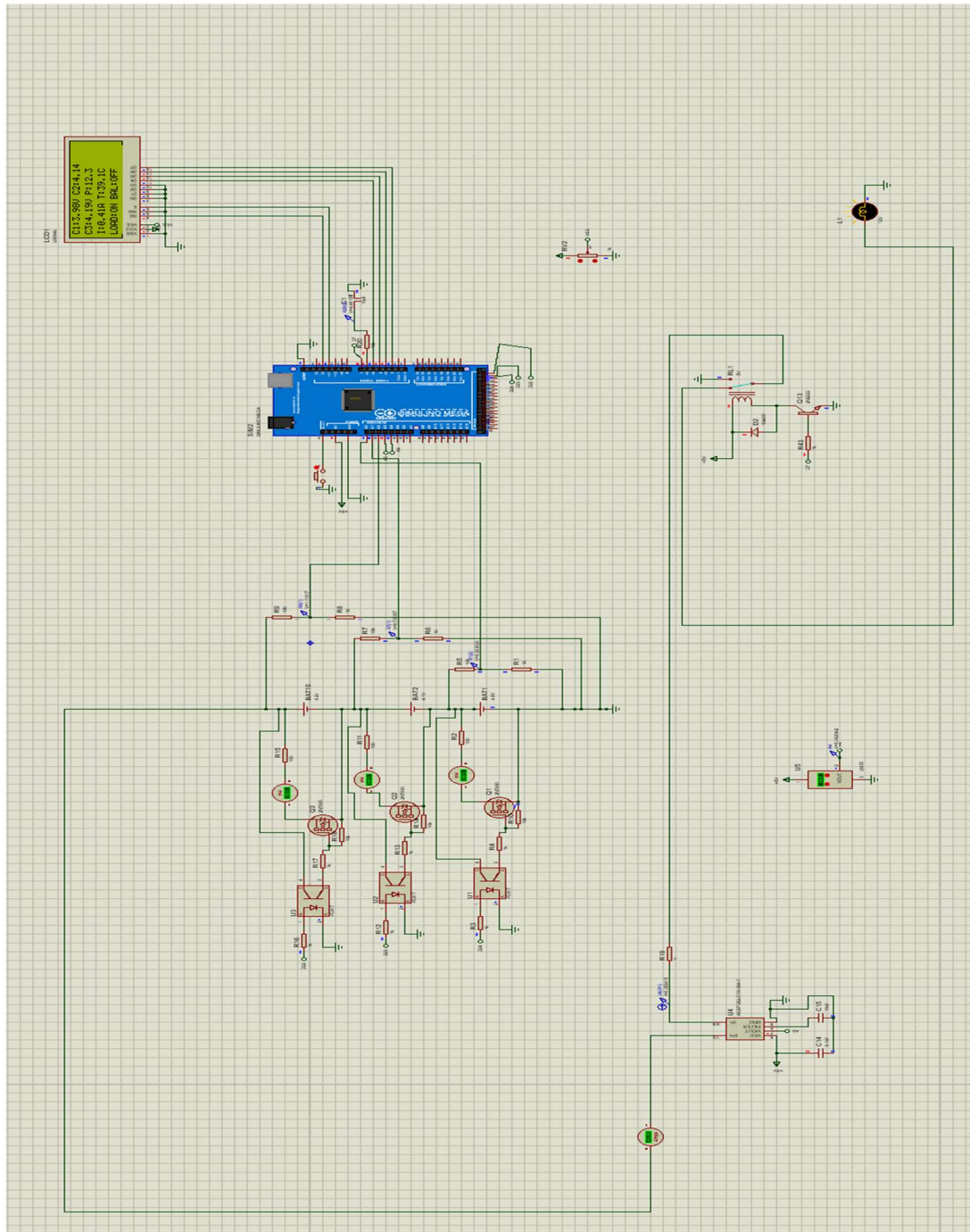


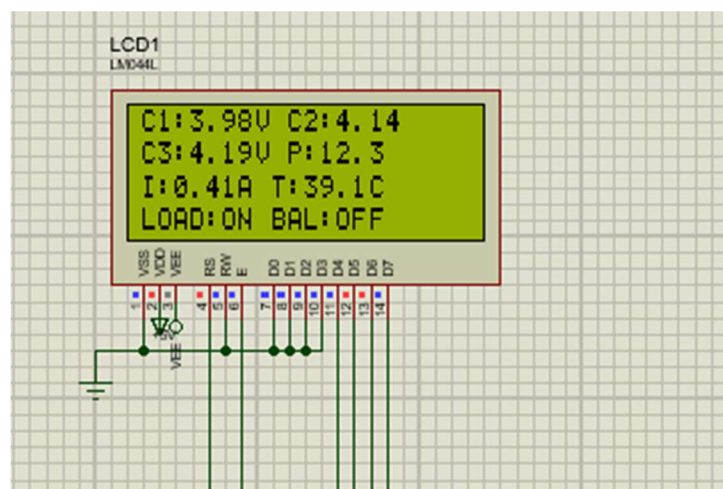
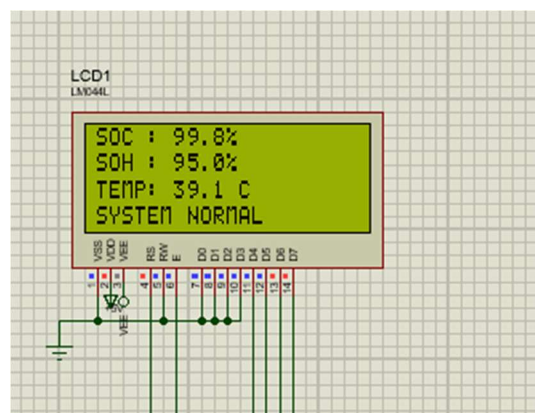
Normal condition

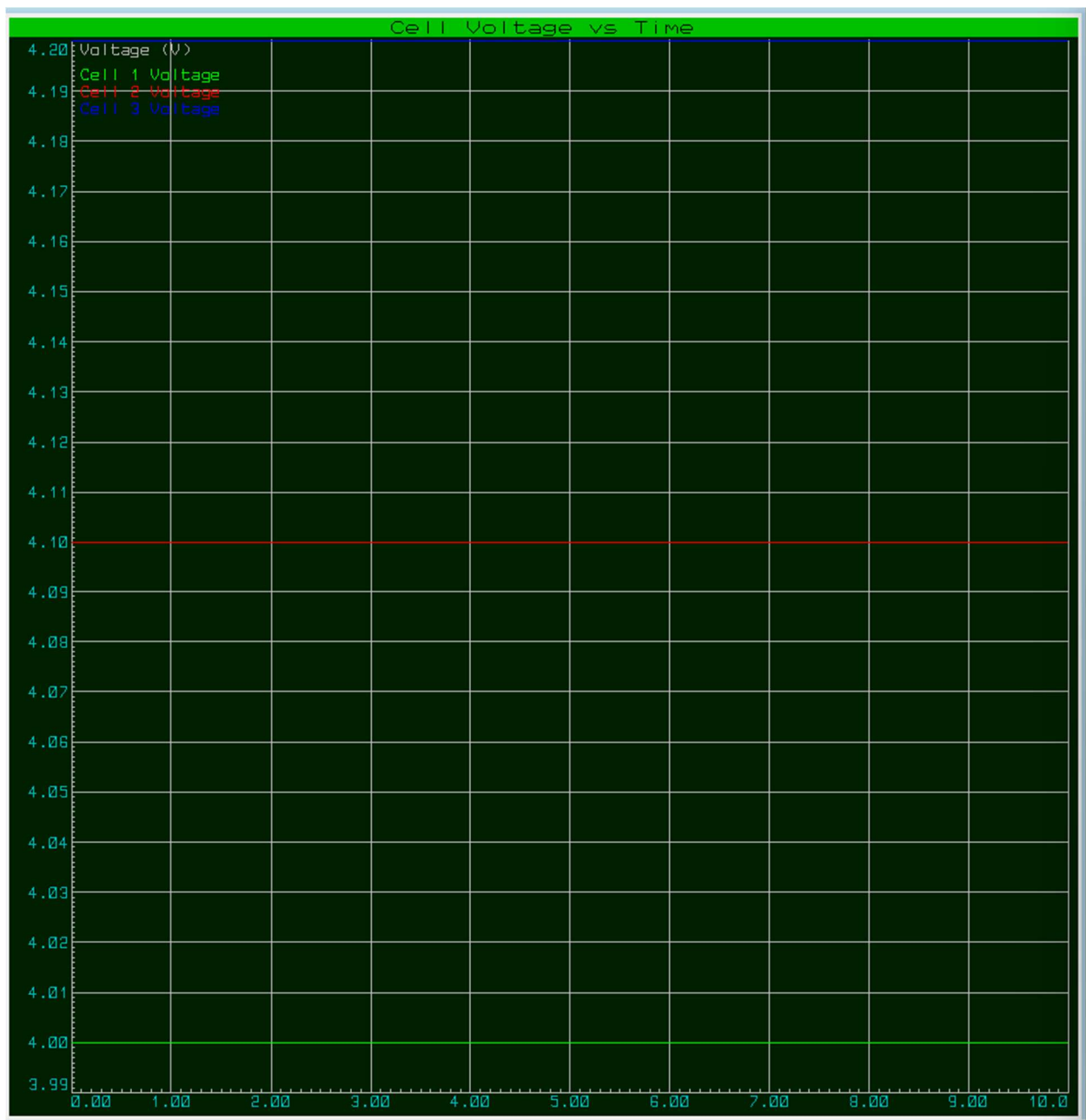
C1= 4.2

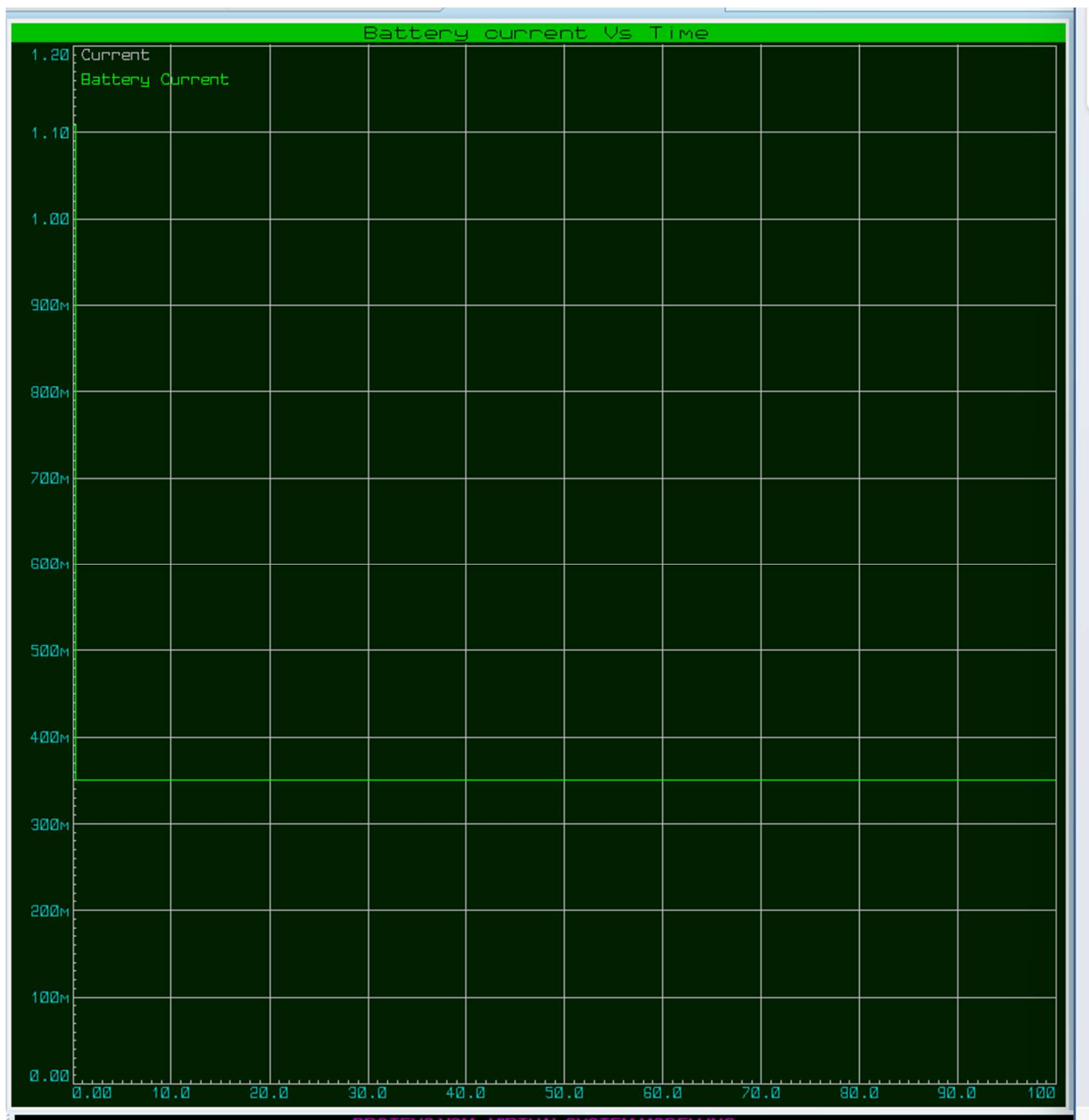
C2=4.1

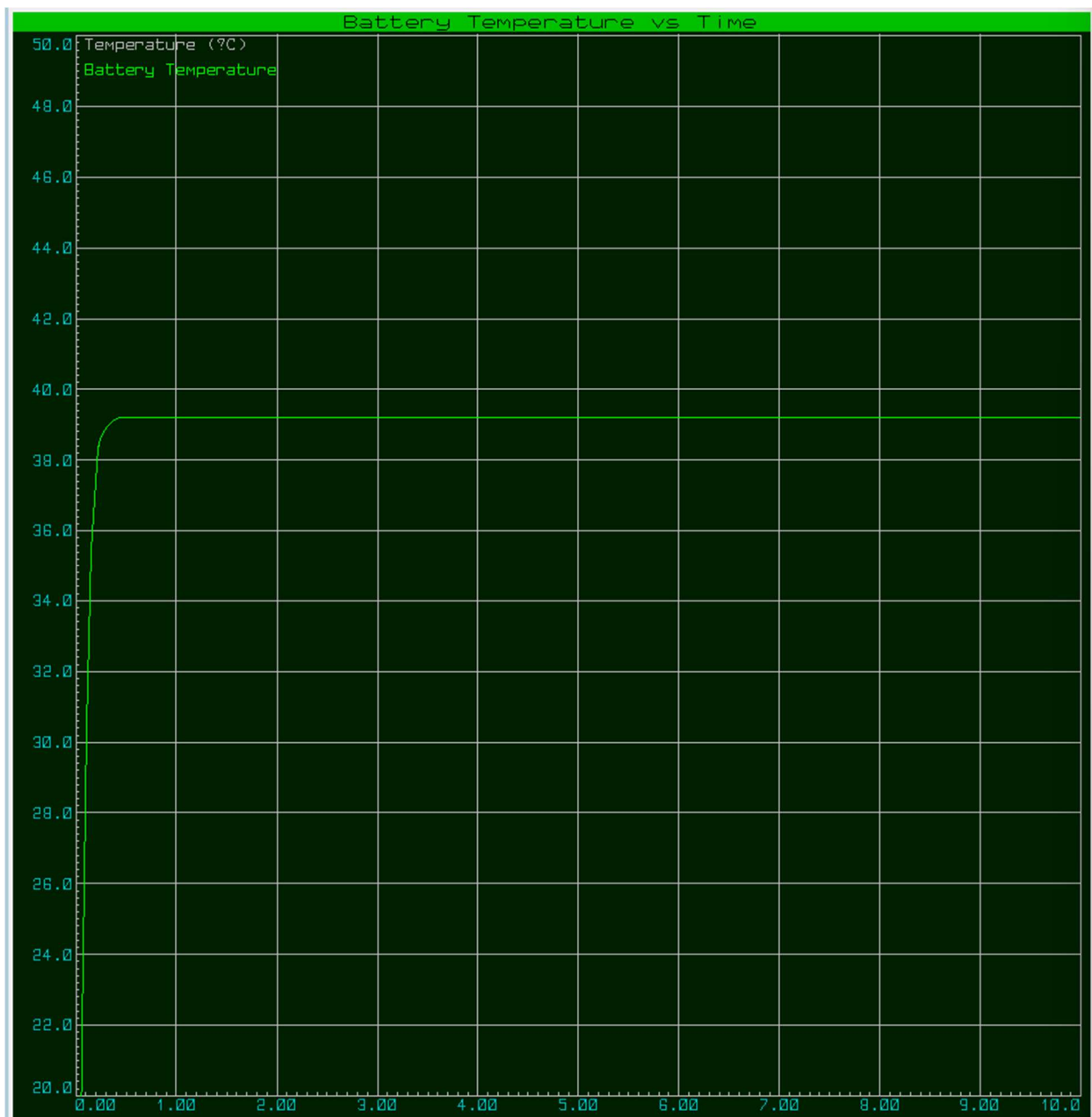
C3=4.0

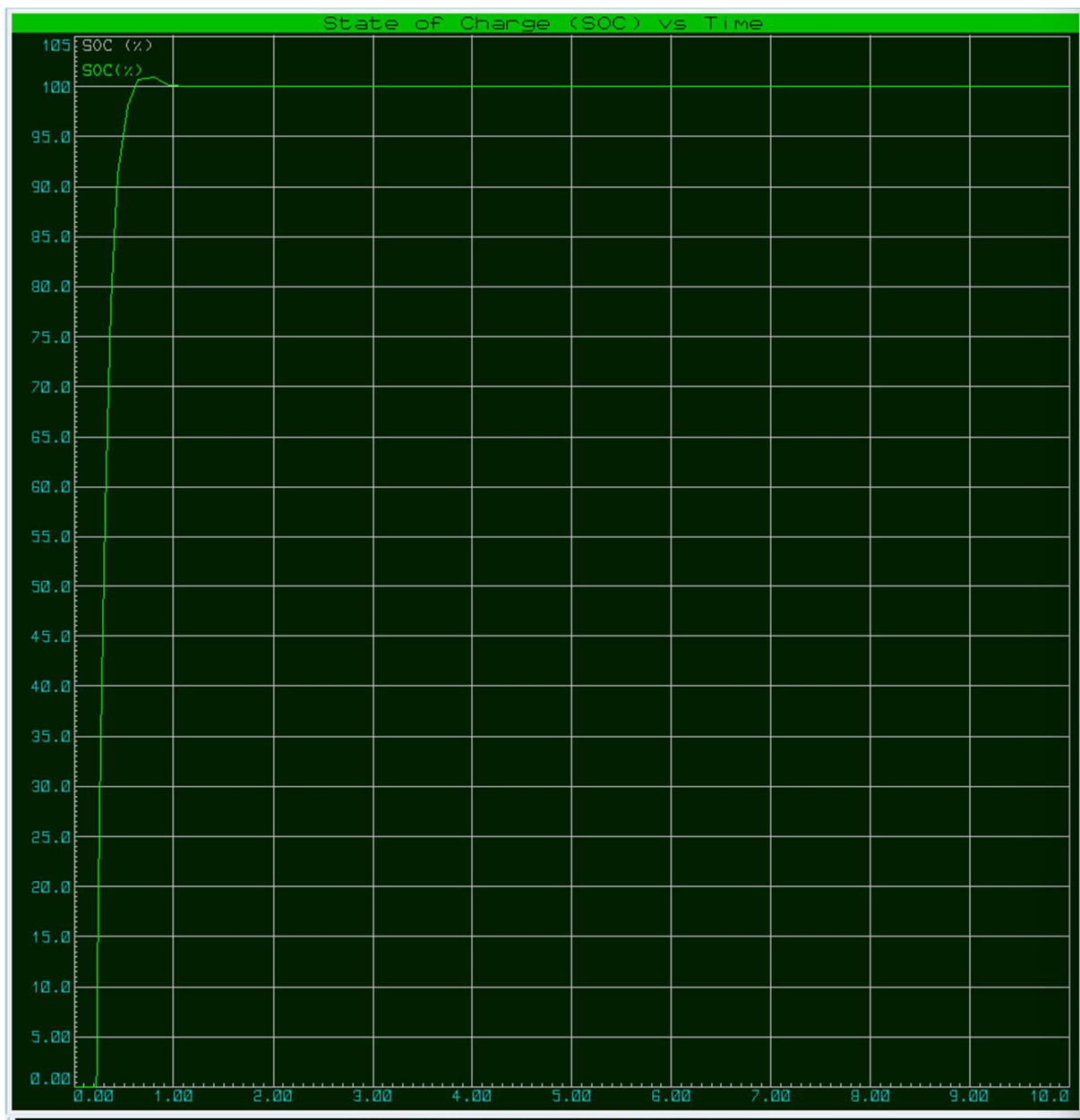










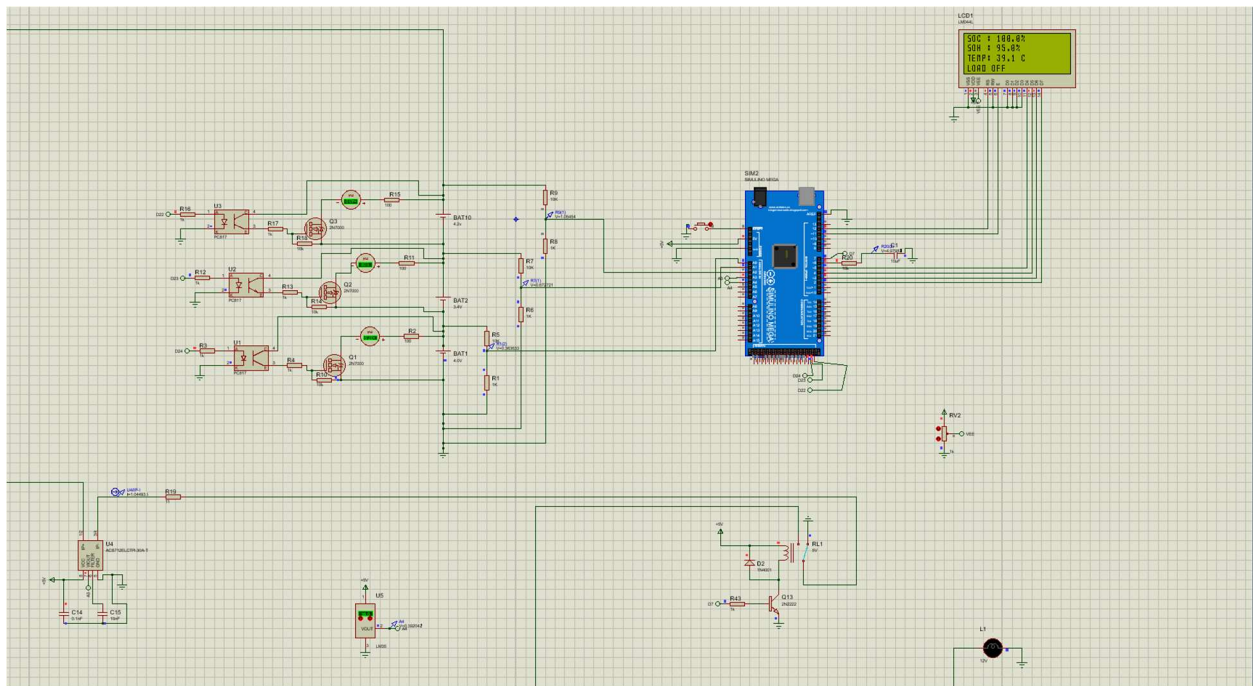


Balancing condition

$C1=4.1$

$C2=3.4$

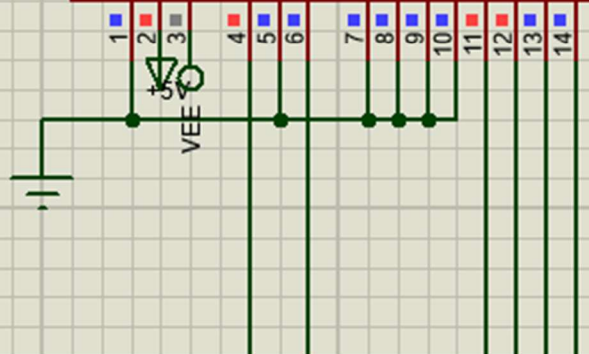
$C3=4.2$



LCD1
LM044L

C1:3.98V C2:3.44
C3:4.19V P:11.6
I:1.07A T:39.1C
BAL:C1 C3

VSS VDD VEE RS RW E D0 D1 D2 D3 D4 D5 D6 D7

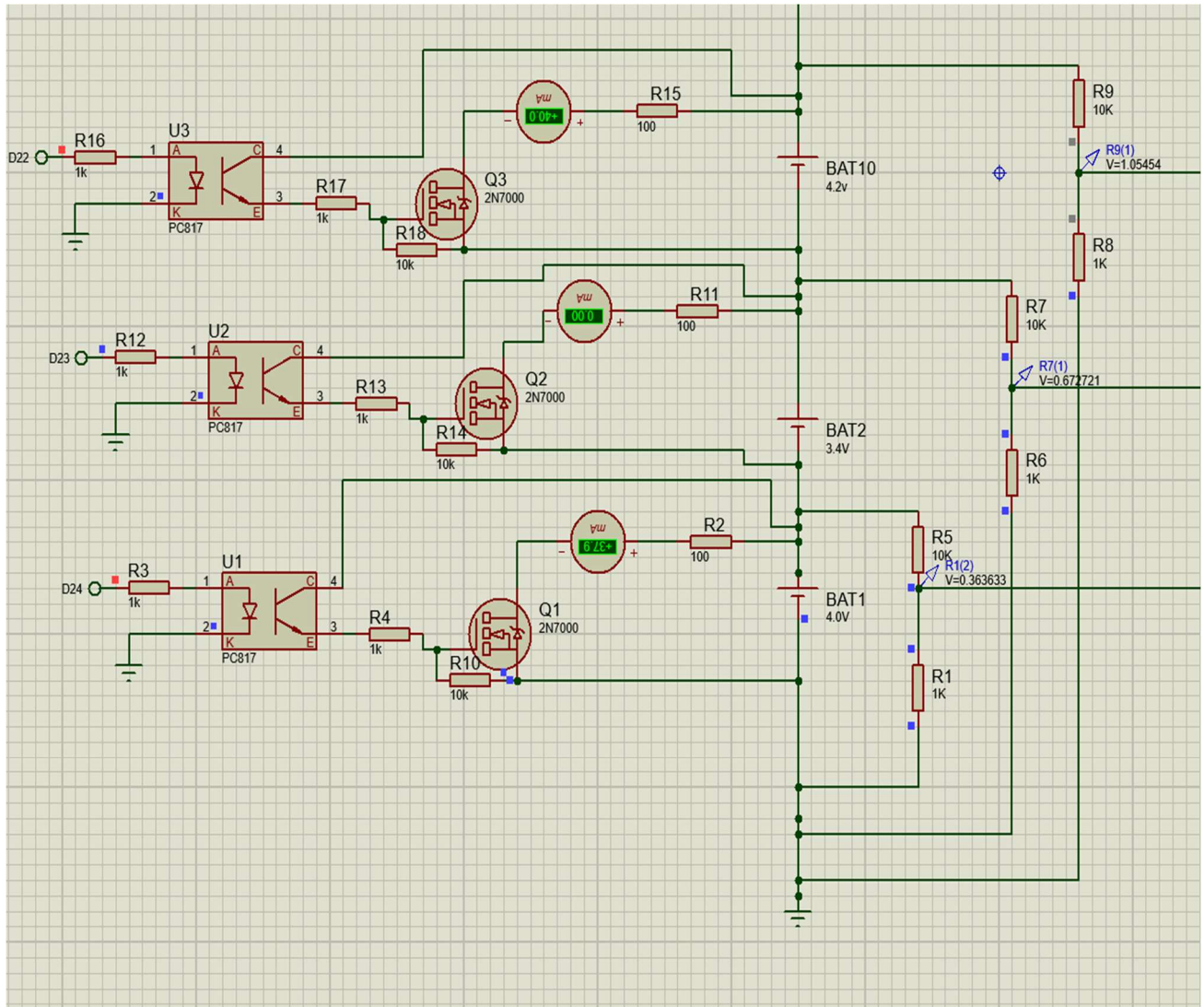


LCD1
LM044L

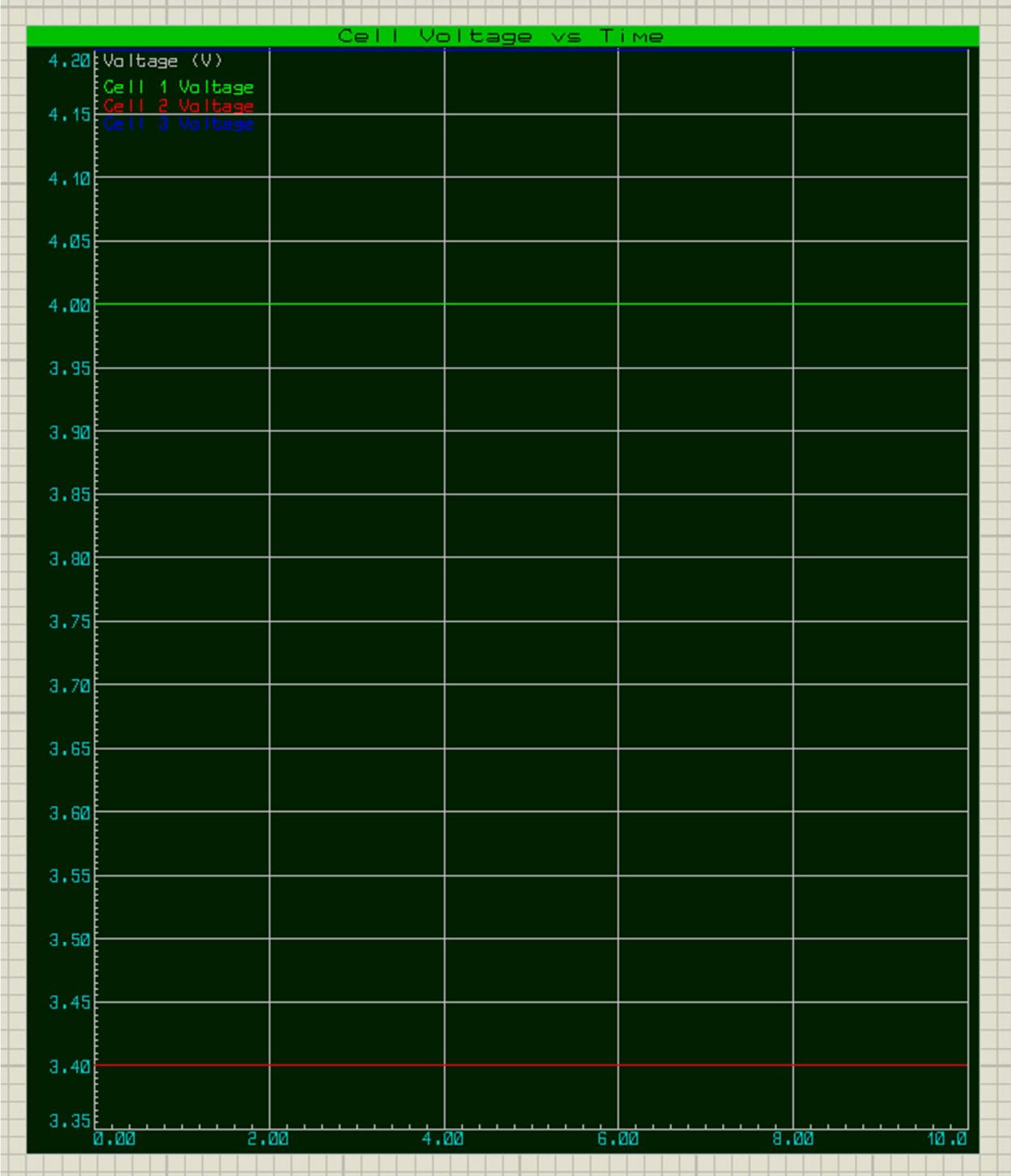
SOC : 99.9%
SOH : 95.0%
TEMP: 39.1 C
LOAD OFF

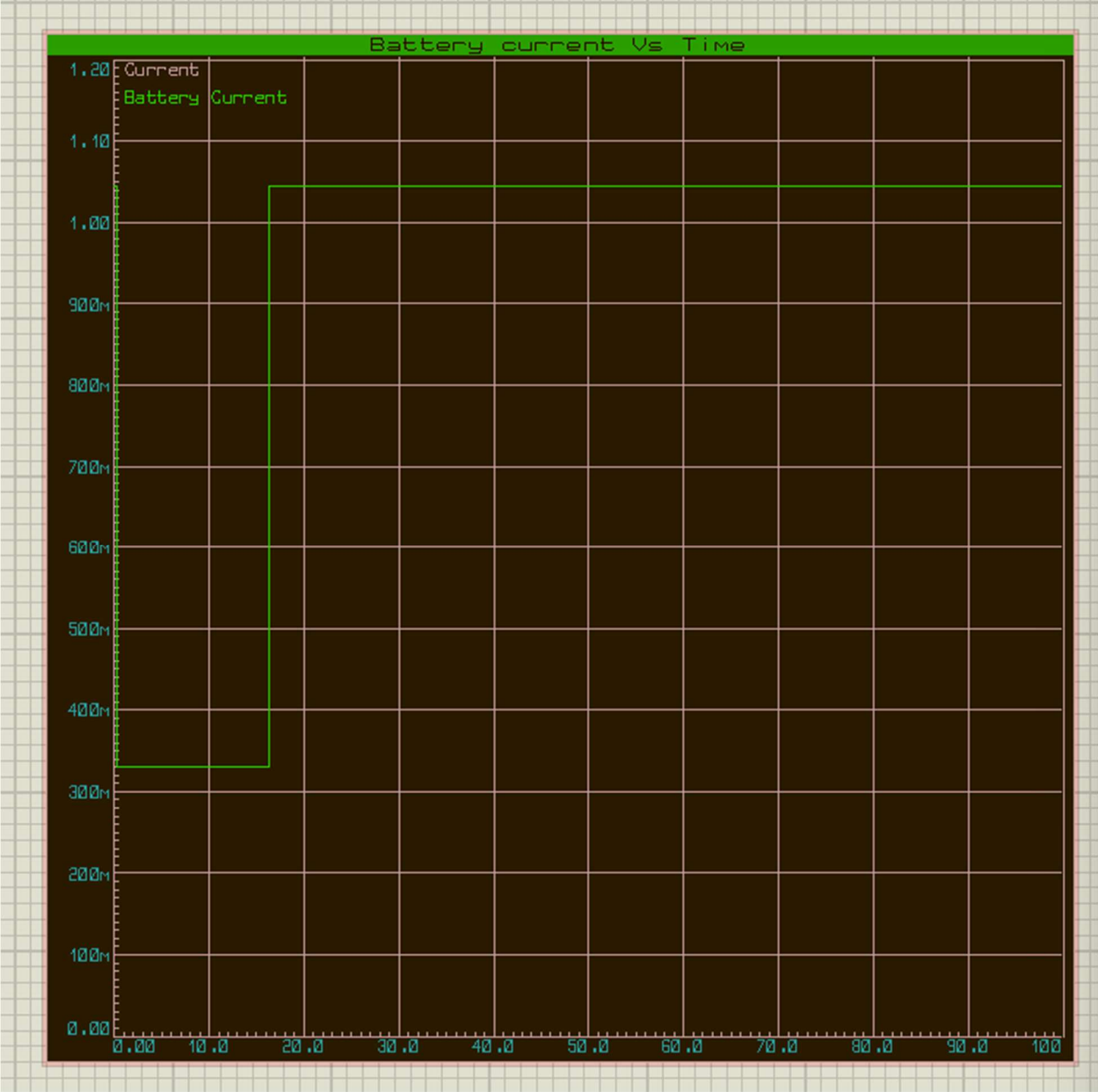
VSS VDD VEE RS RW E D0 D1 D2 D3 D4 D5 D6 D7



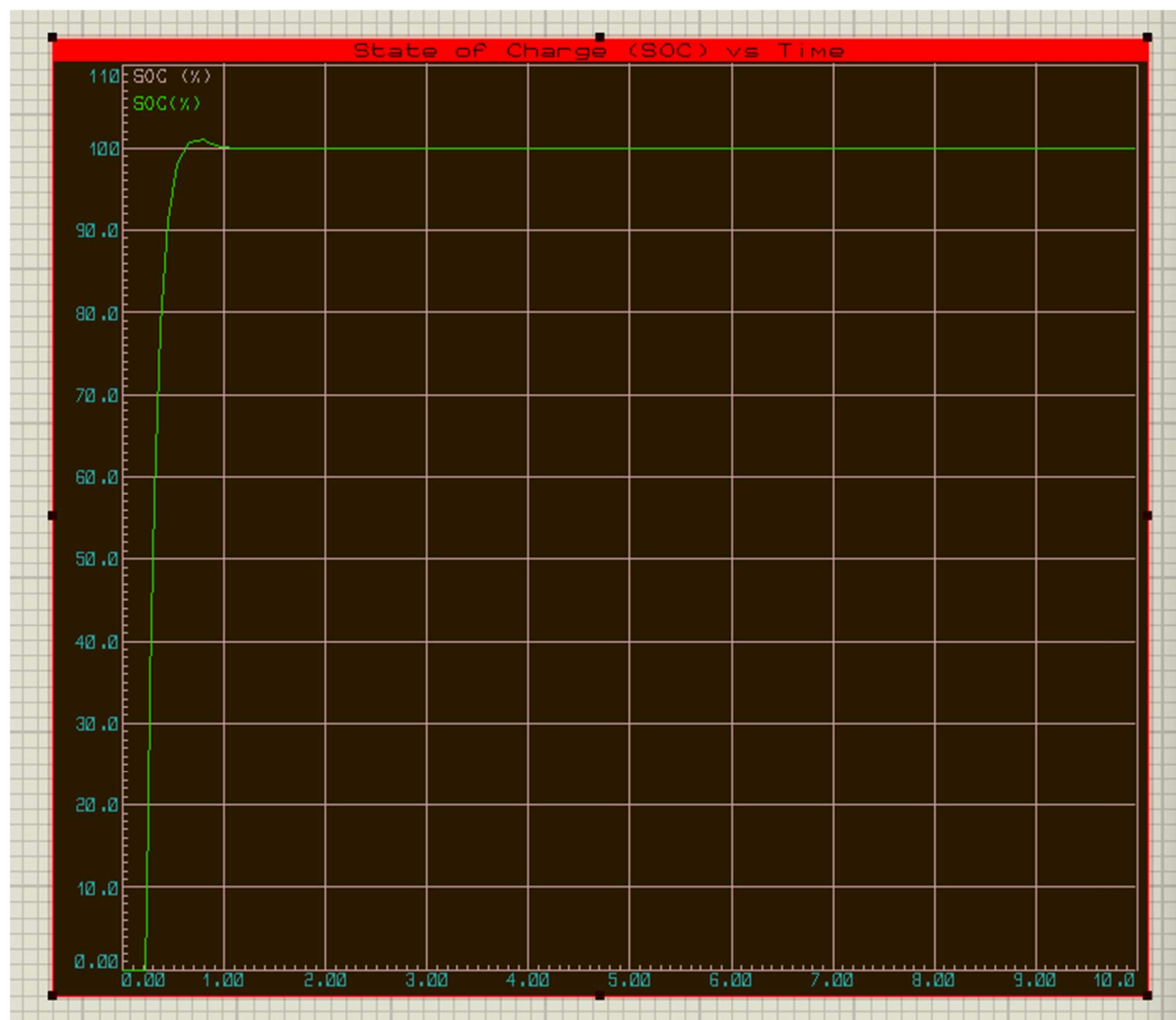


So in this c1 and c2 will loss energy indicated by the ammeter









Code

```
#include <LiquidCrystal.h>
```

```
// =====
```

```
// LCD CONNECTIONS
```

```
// =====
```

```
// RS = D12
```

```
// E = D11
```

```
// D4 = D5
```

```
// D5 = D4
```

```
// D6 = D3
```

```
// D7 = D2
```

```
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
```

```
// =====
```

```
// CELL VOLTAGE MEASUREMENT
```

```
// =====
```

```
#define CELL1_NODE A0
```

```
#define CELL2_NODE A1
```

```
#define PACK_NODE A2
```

```
// =====
```

```
// ACS712 AND LM35
```

```
// =====
```

```
#define CURRENT_PIN A3
```

```
#define TEMP_PIN  A4
```

```
// =====
```

```
// LOAD RELAY
```

```
// =====
```

```
#define RELAY_PIN 7
```

```
// D6 is used only for SOC graph output
```

```
#define SOC_OUTPUT_PIN 6
```

```
// =====
```

```
// PASSIVE BALANCING
```

```
// =====
```

```
// Cell 1 -> D24 -> PC817 -> Q1
```

```
// Cell 2 -> D23 -> PC817 -> Q2
```

```
// Cell 3 -> D22 -> PC817 -> Q3
```

```
#define BAL1_PIN 24
```

```
#define BAL2_PIN 23
```

```
#define BAL3_PIN 22
```

```
// =====
```

```
// VOLTAGE DIVIDER
```

```
// 10k TOP + 1k BOTTOM

// Ratio = (10k + 1k) / 1k = 11

// =====

const float DIVIDER_RATIO = 11.0;

const float ADC_REF = 5.0;

const float ADC_MAX = 1023.0;

// =====

// ACS712-30A-T

// =====

const float ACS_OFFSET = 2.50;

const float ACS_SENSITIVITY = 0.066;

// =====

// BATTERY CAPACITY

// =====

const float RATED_CAPACITY_AH = 2.0;

// Used for SOH demonstration

const float AVAILABLE_CAPACITY_AH = 1.90;
```



```
// =====  
  
// BMS LIMITS  
  
// =====  
  
const float LOW_LIMIT = 3.70;  
  
// Difference required before a cell is bled  
const float BALANCE_DIFF = 0.05;  
  
// =====  
  
// SOC  
  
// =====  
  
float soc = 100.0;  
  
// =====  
  
// TIME FOR COULOMB COUNTING  
  
// =====  
  
unsigned long previousMillis = 0;  
  
// =====  
  
// READ VOLTAGE DIVIDER  
  
// =====
```

```

float readNodeVoltage(int pin)
{
    int adcValue = analogRead(pin);

    float dividerVoltage =
        (adcValue * ADC_REF) / ADC_MAX;

    float actualVoltage =
        dividerVoltage * DIVIDER_RATIO;

    return actualVoltage;
}

// =====
// READ ACS712-30A CURRENT
// =====

float readCurrent()
{
    int adcValue = analogRead(CURRENT_PIN);

    float sensorVoltage =
        (adcValue * ADC_REF) / ADC_MAX;

    float current =
        (sensorVoltage - ACS_OFFSET)

```

```

    / ACS_SENSITIVITY;

    // Remove very small noise
    if (current > -0.05 && current < 0.05)
    {
        current = 0.0;
    }

    return current;
}

// =====
// READ LM35 TEMPERATURE
// =====

float readTemperature()
{
    int adcValue = analogRead(TEMP_PIN);

    float voltage =
        (adcValue * ADC_REF) / ADC_MAX;

    // LM35 = 10 mV / °C
    float temperature =
        voltage * 100.0;

```

```

    return temperature;
}

// =====

// SETUP

// =====

void setup()
{
    lcd.begin(20, 4);

    // SOC PWM output on D6
    pinMode(SOC_OUTPUT_PIN, OUTPUT);

    // Relay
    pinMode(RELAY_PIN, OUTPUT);

    // Balancing PC817 control pins
    pinMode(BAL1_PIN, OUTPUT);
    pinMode(BAL2_PIN, OUTPUT);
    pinMode(BAL3_PIN, OUTPUT);

    // Initial state
    digitalWrite(RELAY_PIN, HIGH);

    digitalWrite(BAL1_PIN, LOW);

```

```
digitalWrite(BAL2_PIN, LOW);  
digitalWrite(BAL3_PIN, LOW);  
  
// Start with 100% SOC  
soc = 100.0;  
  
// 100% SOC = 255 PWM  
analogWrite(SOC_OUTPUT_PIN, 255);  
  
// Startup screen  
lcd.clear();  
  
lcd.setCursor(0, 0);  
lcd.print("3 CELL BMS");  
  
lcd.setCursor(0, 1);  
lcd.print("STATUS: LOADING...");  
  
delay(2000);  
  
lcd.clear();  
  
// Start Coulomb-counting timer  
previousMillis = millis();  
}
```

```

// =====

// MAIN LOOP

// =====

void loop()
{
    // =====

    // 1. READ CUMULATIVE CELL VOLTAGES

    // =====

    float node1 = readNodeVoltage(CELL1_NODE);

    float node2 = readNodeVoltage(CELL2_NODE);

    float packVoltage = readNodeVoltage(PACK_NODE);

    // =====

    // 2. CALCULATE INDIVIDUAL CELL VOLTAGES

    // =====

    float cell1 = node1;

    float cell2 = node2 - node1;

    float cell3 = packVoltage - node2;

```

```

// =====

// 3. FIND LOWEST CELL

// =====

float lowestCell = cell1;

if (cell2 < lowestCell)
{
    lowestCell = cell2;
}

if (cell3 < lowestCell)
{
    lowestCell = cell3;
}

// =====

// 4. LOW CELL DETECTION

// =====

bool lowCell =
    (cell1 <= LOW_LIMIT) ||
    (cell2 <= LOW_LIMIT) ||
    (cell3 <= LOW_LIMIT);

// =====

```

```
// 5. LOAD RELAY CONTROL
```

```
// =====
```

```
if (lowCell)
```

```
{
```

```
    // Any cell reached 3.70 V
```

```
    // Disconnect load
```

```
    digitalWrite(RELAY_PIN, LOW);
```

```
}
```

```
else
```

```
{
```

```
    // Battery okay
```

```
    digitalWrite(RELAY_PIN, HIGH);
```

```
}
```

```
// =====
```

```
// 6. READ CURRENT
```

```
// =====
```

```
float current = readCurrent();
```

```
// =====
```

```
// 7. COULOMB COUNTING
```

```
// =====
```

```
unsigned long currentMillis = millis();
```



```
float deltaTime =  
    (currentMillis - previousMillis) / 3600000.0;  
  
previousMillis = currentMillis;  
  
// Positive current = discharge  
// Negative current = charging  
  
float deltaSOC =  
    (current * deltaTime /  
    RATED_CAPACITY_AH) * 100.0;  
  
// Subtracting positive current decreases SOC.  
// Subtracting negative current increases SOC.  
  
soc = soc - deltaSOC;  
  
// Limit SOC between 0 and 100 %  
  
if (soc > 100.0)  
{  
    soc = 100.0;  
}  
  
if (soc < 0.0)
```

```

{
    soc = 0.0;
}

// =====

// 7A. SOC PWM OUTPUT FOR PROTEUS GRAPH

// =====

int socPWM =

(int)(soc * 255.0 / 100.0);

socPWM =

constrain(socPWM, 0, 255);

analogWrite(SOC_OUTPUT_PIN, socPWM);

// =====

// 8. TEMPERATURE

// =====

float temperature = readTemperature();

// =====

// 9. SOH

// =====

```

```

float soh =

(AVAILABLE_CAPACITY_AH /
RATED_CAPACITY_AH) * 100.0;

// =====

// 10. PASSIVE BALANCING

// =====

bool balanceCell1 = false;

bool balanceCell2 = false;

bool balanceCell3 = false;

// Balancing starts after low-cell protection
//
// The lowest cell is NOT bled.
//
// Any cell more than 0.05 V above
// the lowest cell is bled.

if (lowCell)
{
    if (cell1 > (lowestCell + BALANCE_DIFF))
    {
        balanceCell1 = true;
    }
}

```

```
}
```

```
if (cell2 > (lowestCell + BALANCE_DIFF))
```

```
{
```

```
    balanceCell2 = true;
```

```
}
```

```
if (cell3 > (lowestCell + BALANCE_DIFF))
```

```
{
```

```
    balanceCell3 = true;
```

```
}
```

```
}
```

```
// =====
```

```
// 11. CONTROL PC817 OPTOS
```

```
// =====
```

```
digitalWrite(BAL1_PIN, balanceCell1);
```

```
digitalWrite(BAL2_PIN, balanceCell2);
```

```
digitalWrite(BAL3_PIN, balanceCell3);
```

```
// =====
```

```
// SCREEN 1
```

```
// CELL VOLTAGES + CURRENT + TEMPERATURE
```

```
// =====
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("C1:");
```

```
lcd.print(cell1, 2);
```

```
lcd.print("V ");
```

```
lcd.print("C2:");
```

```
lcd.print(cell2, 2);
```

```
lcd.setCursor(0, 1);
```

```
lcd.print("C3:");
```

```
lcd.print(cell3, 2);
```

```
lcd.print("V ");
```

```
lcd.print("P:");
```

```
lcd.print(packVoltage, 1);
```

```
lcd.setCursor(0, 2);
```

```
lcd.print("I:");
```

```
lcd.print(current, 2);
```

```
lcd.print("A ");
```

```
lcd.print("T:");
```

```
lcd.print(temperature, 1);
```

```
lcd.print("C");
```

```
lcd.setCursor(0, 3);
```

```
// -----
```

```
// LOAD + BALANCING STATUS
```

```
// -----
```

```
if (!lowCell)
```

```
{
```

```
    lcd.print("LOAD:ON BAL:OFF");
```

```
}
```

```
else
```

```
{
```

```
    if (balanceCell1 &&
```

```
        balanceCell2 &&
```

```
        balanceCell3)
```

```
{
```

```
    lcd.print("BAL:C1 C2 C3");
```

```
}
```

```
else if (balanceCell1 &&
```

```
        balanceCell2)
{
    lcd.print("BAL:C1 C2");
}
```

```
else if (balanceCell1 &&
        balanceCell3)
{
    lcd.print("BAL:C1 C3");
}
```

```
else if (balanceCell2 &&
        balanceCell3)
{
    lcd.print("BAL:C2 C3");
}
```

```
else if (balanceCell1)
{
    lcd.print("BAL:C1");
}
```

```
else if (balanceCell2)
{
    lcd.print("BAL:C2");
}
```

```
else if (balanceCell3)
```

```
{
```

```
    lcd.print("BAL:C3");
```

```
}
```

```
else
```

```
{
```

```
    lcd.print("BAL:NONE");
```

```
}
```

```
}
```

```
delay(1500);
```

```
// =====
```

```
// SCREEN 2
```

```
// SOC + SOH + TEMPERATURE
```

```
// =====
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("SOC : ");
```

```
lcd.print(soc, 1);
```

```
lcd.print("%");
```



```
lcd.setCursor(0, 1);
```

```
lcd.print("SOH : ");
```

```
lcd.print(soh, 1);
```

```
lcd.print("%");
```

```
lcd.setCursor(0, 2);
```

```
lcd.print("TEMP: ");
```

```
lcd.print(temperature, 1);
```

```
lcd.print(" C");
```

```
lcd.setCursor(0, 3);
```

```
// No charging/discharging mode
```

```
// Just show overall system status
```

```
if (temperature >= 45.0)
```

```
{
```

```
    lcd.print("TEMP HIGH");
```

```
}
```

```
else if (lowCell)
```

```
{
```

```
    lcd.print("LOAD OFF");
```

```
}
```

```
else
```

```
{
```

```
  lcd.print("SYSTEM NORMAL");
```

```
}
```

```
delay(2000);
```

```
}
```